

NEURONWORKS / LEARNING SERIES

INTERMEDIATE PROGRAMMER JAVASCRIPT

Reliable Runtime Reasoning, Testable Architecture,
React, TypeScript, and Next.js

TECHNICAL LEARNING BOOK — V2

PT Neuronworks Indonesia

Program Pengembangan Kompetensi Programmer | 2026

Edisi 2.0 / Agustus 2026

Panduan Penggunaan Modul

Posisi modul: Modul ini memvalidasi kemampuan programmer project dalam memahami perilaku runtime, mengelola state dan proses asynchronous, memisahkan tanggung jawab kode, serta membangun aplikasi yang dapat diuji dan dirawat.

Komponen	Ketentuan
Level	Intermediate Programmer
Durasi	1 minggu; estimasi 12–16 jam self-learning di luar mentoring dan project
Prasyarat	JavaScript fundamental, DOM, HTTP dasar, Git, terminal, npm, dan HTML/CSS Intermediate
Perangkat	Browser/DevTools, Node.js/npm, editor, repository latihan, test runner
Metode	Prediksi → jalankan → observasi → uji → refactor → jelaskan trade-off
Bukti	Source code, test report, network/runtime evidence, profiling note, dan technical reasoning

Cara belajar

- Prediksi output dan urutan eksekusi sebelum menjalankan kode.
- Pisahkan fakta runtime dari asumsi framework.
- Buat failing test atau reproduksi minimum sebelum memperbaiki bug.
- Uji happy path, error, loading, cancellation, data kosong, dan interaksi berulang.
- Refactor hanya setelah behavior terkunci dan alasan perubahan dapat dijelaskan.

Batas kedalaman

Modul tidak menuntut hafalan seluruh API React/Next.js atau penggunaan satu state manager untuk semua kasus. Fokusnya adalah model mental JavaScript, reliability, testability, separation of concerns, serta keputusan arsitektur yang proporsional. API dan versi library harus mengikuti package lock serta dokumentasi resmi project.

Peta Pembelajaran V2

CARA MEMBACA MODUL		
Jalur Core memuat 35 topik wajib JavaScript pada silabus Intermediate. Supporting menjelaskan praktik yang membuat implementasi Core reliable, secure, testable, dan maintainable. Reference digunakan sesuai versi dan kebutuhan project, bukan sebagai target hafalan.		
Label	Makna	Perlakuan belajar
CORE	Langsung berasal dari silabus Intermediate.	Wajib dipahami, dipraktikkan, dan dapat dijelaskan.
SUPPORTING	Mendukung reliability, security, testing, atau maintainability.	Pelajari melalui failure mode dan keputusan implementasi.
REFERENCE	API, library, atau variasi implementasi sesuai versi project.	Gunakan dokumentasi resmi saat relevan; tidak dihafalkan.

Learning Path Satu Minggu

Hari	Fokus	Bab	Evidence minimum
1	Runtime reasoning	1–2	Prediksi, failing test, stack/coercion trace
2	Event, async, dan Web API	3–5	Timeline, listener/network/storage evidence
3	Testing, TypeScript, dan boundary	6–8	Test report, type contract, service boundary
4	React dan Next.js	9–11	State/effect trace, component test, route/auth evidence
5	Refactoring dan profiling	12	Evidence pack, before-after metric, technical defense

Bukti minimum peserta

- Source code, package lock, dan instruksi setup/build yang dapat direproduksi.
- Prediksi output atau timeline sebelum eksekusi, lalu bukti runtime untuk memvalidasinya.
- Automated test yang mencakup normal, error, empty, repeated interaction, dan regression path yang relevan.
- DevTools evidence: console/stack, event, network, storage, render, atau profiler sesuai kasus.
- Decision record yang menjelaskan requirement, alternatif, trade-off, risiko, dan alasan pilihan.

Ringkasan Traceability

Kelompok silabus	Topik	Bab	Bukti utama
Variables, operators, functions	7	1–2	Prediksi, identity/coercion, stack trace
Event dan asynchronous	6	3–4	Event flow, timeline, cancellation/error test
Web API, error, interaction	3	5	Storage/error/dialog decision evidence
Testing dan TypeScript	4	6–7	Jest/Cypress report dan type/runtime contract
Code structure dan React	9	8–10	Boundary, state/effect, form/component tests
Next.js	4	11	Route, rendering, layout, dan auth evidence
Critical thinking	2	12	Profiling, refactor, dan before-after metric

Catatan normalisasi silabus

NORMALISASI
Istilah pada sumber seperti 'Asynchronous Javascript', 'Typescript', 'Component Lifecycle', 'Conditional rendering', 'Layouting', dan 'Authenticate' ditampilkan dengan ejaan teknis yang lazim. 'Component Test Cypress' dipahami sebagai Cypress Component Testing. Normalisasi hanya memperjelas istilah dan tidak mengubah 35 cakupan topik.

Capaian Pembelajaran

- menjelaskan value/reference, mutation, identity, destructuring, dan operator lanjutan;
- menggunakan arrow function, closure, IIFE, recursion, dan stack secara tepat;
- membangun event handling yang aman melalui binding dan delegation;
- mengelola timer, callback, promise, async/await, concurrency, error, dan cancellation;
- menggunakan Web Storage dan Fetch tanpa mengabaikan keamanan serta lifecycle data;
- menulis unit test Jest dan component test Cypress yang berorientasi behavior;
- menggunakan TypeScript type dan function type untuk memperjelas kontrak;
- memisahkan UI, business logic, state, dan API access;
- mengimplementasikan React/Redux/React Hook Form dan Next.js App Router secara terukur; dan
- menentukan prioritas refactoring serta membaca bukti profiling sebelum optimasi.

Peta Kompetensi

Area	Kemampuan inti	Bukti
Runtime reasoning	Identity, scope, stack, event loop, async flow	Prediksi output dan trace konsisten
Browser interaction	Event, storage, fetch, error/cancellation	State UI benar pada semua jalur
Testability	Unit/component test berbasis behavior	Test deterministik dan bermakna
Type safety	Type, union, narrowing, function contract	Compile error mencegah defect
Architecture	UI–logic–service boundary	Dependency dan ownership jelas
React/Next	State, effect, context/store, routing/rendering/auth	Aplikasi dapat dibangun dan diuji

Area	Kemampuan inti	Bukti
Engineering judgement	Refactoring dan profiling	Decision record dan evidence

Komposisi Pembelajaran

Aktivitas	Proporsi	Fokus
Runtime/concept	25%	Mental model dan kontrak
Implementation	30%	Browser, React, Next.js
Testing/debugging	30%	Evidence dan reliability
Project/reasoning	15%	Integrasi dan keputusan

Isi Buku

- 01 Values, references, dan destructuring
- 02 Operators, functions, recursion, dan stack
- 03 Browser events dan event delegation
- 04 Asynchronous JavaScript yang dapat diandalkan
- 05 Web APIs, storage, interaction, dan error strategy
- 06 Testable JavaScript dengan Jest dan Cypress
- 07 TypeScript sebagai kontrak
- 08 Separation of concerns dan service layer
- 09 React fundamentals dan component behavior

10 React state, Context/Redux, dan forms

11 Next.js App Router, rendering, layout, dan authentication

12 Refactoring, profiling, dan mini-project

BAB 01

Values, References, dan Destructuring

Mengendalikan mutation dan identity agar data flow tidak menghasilkan side effect tersembunyi.

Klasifikasi	CORE
Fokus silabus	More variables — value vs reference dan destructuring assignments.
Project case	Perubahan pada data hasil salin ikut mengubah state asal dan menghasilkan bug yang sulit direproduksi. Peserta harus menjelaskan identity, mutation, serta batas salinan yang dibuat.
Bukti Intermediate	Prediksi output, identity check, failing test, dan diff refactoring tanpa mutation tersembunyi.
Best practice	Bedakan value, reference, dan identity sebelum memilih spread, destructuring, structuredClone, atau strategi immutable lain.

TARGET KOMPETENSI

- Membedakan primitive value dari reference identity.
- Membuat update immutable yang cukup dalam.
- Menggunakan destructuring, rest, spread, dan default secara aman.

1.1 Value, reference, dan identity

Primitive disalin sebagai nilai. Object, array, dan function disimpan serta dibandingkan melalui reference identity. Assignment tidak menggandakan object; dua variabel dapat menunjuk object yang sama. `const` mencegah reassignment binding, bukan membuat isi object immutable.

Contoh 1. Shallow copy belum memutus nested reference

```
const original = { id: 7, meta: { status: 'open' } };
const alias = original;
const shallow = { ...original };

alias.meta.status = 'closed';
console.log(original.meta.status); // closed
console.log(shallow.meta === original.meta); // true
```

Operasi	Identity baru	Nested aman?
const b = a	Tidak	Tidak
{ ...a } / [...a]	Top-level	Tidak
structuredClone(a)	Ya	Umumnya ya; perhatikan tipe yang didukung
Domain mapper	Sesuai kontrak	Dapat dikendalikan eksplisit

1.2 Destructuring sebagai kontrak lokal

Contoh 2. Destructuring, rest, dan immutable update

```
function summarizeProject({ id, name, risk = 'unknown', owner: { name: ownerName } = {} }) {
  return { id, label: `${name} - ${ownerName ?? 'Unassigned'}`, risk };
}

const [first, ...remaining] = projects;
const next = projects.map(p => p.id === changed.id ? { ...p, ...changed } : p);
```

Waspada: Spread adalah shallow copy. Jangan menganggap {...state} otomatis memutus seluruh nested reference atau aman untuk semua object khusus.

RED FLAG — KESALAHAN UMUM

- Mengubah object input di dalam helper tanpa kontrak mutation.
- Membandingkan dua object berdasarkan isi dengan ===.
- Menggunakan JSON parse/stringify sebagai clone universal.
- Destructuring terlalu dalam sehingga kode rapuh terhadap perubahan shape.

PRAKTIK & EVIDENCE 1

Debug bug filter project yang mengubah data sumber. Tulis test identity, refactor menjadi update immutable, lalu jelaskan bagian mana yang tetap berbagi reference.

Definition of done: Input tidak berubah; output identity sesuai kebutuhan; nested update aman; test membuktikan behavior dan peserta dapat menjelaskan shallow versus deep copy.

CHECKPOINT Bab 1

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Mengapa const object masih dapat berubah?

Apa yang disalin spread pada object?

Kapan destructuring default dijalankan?

BAB 02

Operators, Functions, Recursion, dan Stack

Menggunakan sintaks ringkas tanpa mengaburkan coercion, control flow, this, atau batas stack.

Klasifikasi	CORE
Fokus silabus	More operators/functions — relational, bitwise, string, comma, unary, instanceof, arrow function, IIFE, recursion, dan stack.
Project case	Validasi menghasilkan hasil berbeda karena coercion, callback kehilangan context, dan proses rekursif gagal pada data besar.
Bukti Intermediate	Tabel prediksi operator, reproduksi context, stack trace, guard condition, dan alternatif iteratif.
Best practice	Gunakan operator dan bentuk fungsi berdasarkan kontrak data serta kebutuhan context; batasi recursion dengan base case dan analisis kedalaman.

TARGET KOMPETENSI

- Menilai relational, unary, string, bitwise, comma, dan instanceof secara kontekstual.
- Menjelaskan lexical this pada arrow function dan kegunaan IIFE.
- Mendesain recursion dengan base case dan alternatif iterative.

2.1 Operator dan coercion

Operator	Gunakan	Risiko
=== / !==	Perbandingan tanpa coercion implisit	Tetap perhatikan NaN dan object identity
??	Fallback hanya null/undefined	Berbeda dari untuk 0, false, dan string kosong

Operator	Gunakan	Risiko
?.	Akses/-call opsional	Dapat menutupi kontrak data yang seharusnya wajib
instanceof	Relasi prototype dalam realm sesuai	Tidak cocok untuk semua validasi data
bitwise	Flag/operasi integer terukur	Konversi 32-bit dan readability
comma	Ekspresi khusus yang sangat terbatas	Mudah disalahbaca

Contoh 3. Operator dengan intent jelas

```
const retries = config.retries ?? 3; // 0 tetap dipertahankan
const label = project.owner?.name ?? 'Unassigned';

class DomainError extends Error {}
console.log(error instanceof DomainError);

const CAN_READ = 1 << 0;
const CAN_WRITE = 1 << 1;
const access = CAN_READ | CAN_WRITE;
```

2.2 Arrow function, this, dan IIFE

Arrow function tidak memiliki this, arguments, atau prototype sendiri. Ia cocok untuk callback yang memakai lexical scope, tetapi bukan pengganti otomatis method yang memerlukan dynamic this. IIFE dapat membuat scope satu kali, walau ES modules kini sering menjadi boundary yang lebih jelas.

Contoh 4. Method dan lexical this

```
const controller = {
  value: 0,
  increment() { this.value += 1; },
  delayedIncrement() {
    setTimeout(() => { this.value += 1; }, 100);
  }
};

(() => {
  const internalFlag = true;
  initialize(internalFlag);
})();
```

2.3 Recursion dan stack

Contoh 5. Recursion dengan base case

```
function findNode(node, targetId) {
  if (!node) return null;           // base case
  if (node.id === targetId) return node;
  for (const child of node.children ?? []) {
    const found = findNode(child, targetId);
    if (found) return found;
  }
  return null;
}
```

Engineering choice: Recursion natural untuk struktur tree, tetapi kedalaman input harus diketahui. Untuk data sangat dalam atau tidak tepercaya, pertimbangkan stack eksplisit dan batas proses.

PRAKTIK & EVIDENCE 2

Prediksi output lima ekspresi operator, perbaiki method arrow yang kehilangan this, lalu ubah traversal rekursif menjadi iterative untuk input dalam.

Definition of done: Tidak ada coercion tak sengaja; this benar; traversal punya termination; solusi iterative diuji pada data besar dan trade-off tercatat.

CHECKPOINT Bab 2

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa beda ?? dan || untuk nilai 0?

Mengapa arrow function tidak selalu tepat sebagai method?

Apa dua syarat recursion yang aman?

BAB 03

Browser Events dan Event Delegation

Mengelola interaksi dinamis tanpa listener berlebihan, memory leak, atau perilaku default yang tidak disengaja.

Klasifikasi	CORE
Fokus silabus	Event — browser event, binding, dan event delegation.
Project case	Tombol pada baris dinamis tidak merespons, sedangkan handler terpasang berulang dan mengeksekusi aksi ganda.
Bukti Intermediate	Event-flow trace, listener inventory, reproduksi elemen dinamis, dan regression test interaksi berulang.
Best practice	Pilih binding langsung atau delegation berdasarkan lifecycle elemen; filter target dan kelola cleanup listener secara eksplisit.

TARGET KOMPETENSI

- Menjelaskan capture, target, bubble, dan default action.
- Menggunakan addEventListener serta cleanup yang tepat.
- Menerapkan delegation dengan closest() dan boundary check.

3.1 Event flow dan binding

Browser mendispatch event melalui capture menuju target, lalu bubble untuk event yang mendukungnya. Handler menerima event object; currentTarget adalah elemen pemilik listener, sedangkan target adalah sumber awal. preventDefault membatalkan aksi default, bukan menghentikan propagation.

Contoh 6. Binding dengan cleanup

```
const controller = new AbortController();

document.querySelector('#filters').addEventListener('submit', (event) => {
  event.preventDefault();
  applyFilters(new FormData(event.currentTarget));
}, { signal: controller.signal });

// lifecycle cleanup
controller.abort();
```

3.2 Delegation untuk elemen dinamis

Contoh 7. Event delegation

```
const list = document.querySelector('[data-project-list]');

list.addEventListener('click', (event) => {
  const button = event.target.closest('[data-action]');
  if (!button || !list.contains(button)) return;

  const row = button.closest('[data-project-id]');
  if (button.dataset.action === 'archive') archiveProject(row.dataset.projectId);
});
```

Masalah	Pemeriksaan
Handler berjalan dua kali	Listener terpasang ulang atau event bubble ke handler lain
Klik icon tidak terbaca	target adalah child; gunakan closest pada selector aksi
Link tidak berpindah	preventDefault dipanggil tanpa kondisi
Memory meningkat	Listener/timer tidak dibersihkan saat lifecycle berakhir

RED FLAG — KESALAHAN UMUM

- Menggunakan onclick inline untuk semua interaksi.
- Menyamakan target dan currentTarget.
- Memanggil stopPropagation untuk menutupi desain event yang tidak jelas.
- Delegation tanpa memastikan target masih berada dalam container.

PRAKTIK & EVIDENCE 3

Bangun tabel project dinamis dengan aksi view, retry, dan archive melalui satu delegated listener. Tambahkan cleanup dan uji klik pada text/icon di dalam button.

Definition of done: Satu listener melayani row baru; aksi tepat; tidak memblokir default action lain; cleanup terbukti; keyboard activation tetap bekerja pada elemen semantik.

CHECKPOINT Bab 3

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa beda target dan currentTarget?

Mengapa delegation cocok untuk item dinamis?

Apa yang tidak dilakukan preventDefault()?

BAB 04

Asynchronous JavaScript yang Dapat Diandalkan

Menentukan urutan kerja, error boundary, concurrency, dan cancellation tanpa callback chaos atau race condition.

Klasifikasi	CORE
Fokus silabus	Asynchronous JavaScript — setTimeout, setInterval, callback, Promise, serta async/await.
Project case	Respons request lama menimpa data terbaru, interval terus berjalan setelah halaman berubah, dan error Promise tidak tertangani.
Bukti Intermediate	Timeline eksekusi, network trace, cancellation/cleanup, error-path test, dan bukti race condition telah dicegah.
Best practice	Modelkan operasi async sebagai state yang memiliki success, error, timeout, cancellation, cleanup, dan potensi hasil datang tidak berurutan.

TARGET KOMPETENSI

- Membedakan synchronous stack, task, dan promise reaction.
- Mengubah callback menjadi promise/async flow yang dapat ditangani.
- Memilih sequential, parallel, race, dan cancellation sesuai requirement.

4.1 Timer bukan jaminan waktu tepat

Contoh 8. Urutan eksekusi

```
console.log('A');
setTimeout(() => console.log('timer'), 0);
Promise.resolve().then(() => console.log('promise'));
console.log('B');

// A, B, promise, timer
```

setTimeout menjadwalkan callback setelah delay minimum; callback tetap menunggu stack dan antrean terkait. setInterval dapat menumpuk pekerjaan bila proses lebih lama dari interval. Untuk polling, recursive timeout setelah pekerjaan selesai sering lebih mudah dikendalikan.

4.2 Promise dan async/await

Contoh 9. Parallel independent work dan HTTP error

```
async function loadDashboard(signal) {
  const [projects, risks] = await Promise.all([
    fetchJSON('/api/projects', { signal }),
    fetchJSON('/api/risks', { signal }),
  ]);
  return { projects, risks };
}

async function fetchJSON(url, options) {
  const response = await fetch(url, options);
  if (!response.ok) throw new HttpError(response.status, url);
  return response.json();
}
```

Kebutuhan	Teknik
Semua independen harus sukses	Promise.all
Butuh hasil setiap operasi	Promise.allSettled
Ambil yang pertama settle	Promise.race
Ambil yang pertama fulfill	Promise.any
Batalkan request usang	AbortController/AbortSignal
Harus berurutan	await sequential dengan alasan dependency

4.3 Error, finally, dan race condition

Contoh 10. Membatalkan request usang

```
let activeController;

async function search(query) {
  activeController?.abort();
  activeController = new AbortController();
  setLoading(true);
  try {
    return await fetchJSON(`/api/search?q=${encodeURIComponent(query)}`,
      { signal: activeController.signal });
  } catch (error) {
    if (error.name !== 'AbortError') showError(error);
  } finally {
    setLoading(false);
  }
}
```

RED FLAG — KESALAHAN UMUM

- Lupa return/await promise sehingga error terlepas.

- Memakai await di loop untuk pekerjaan yang sebenarnya independen.
- Menganggap fetch reject pada setiap HTTP 4xx/5xx.
- Mengubah state loading dari request lama setelah request baru selesai.

PRAKTIK & EVIDENCE 4

Bangun live search dengan debounce, cancellation, loading, empty, success, dan error state. Tambahkan test urutan respons lambat–cepat.

Definition of done: Request lama dibatalkan/diabaikan; HTTP error ditangani; loading selalu selesai; tidak ada stale result; test memakai timer/network control yang deterministik.

CHECKPOINT Bab 4

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Mengapa setTimeout(...,0) tidak langsung berjalan?

Kapan Promise.all tidak tepat?

Bagaimana cancellation mengurangi race condition?

BAB 05

Web APIs, Storage, Interaction, dan Error Strategy

Menggunakan API browser sesuai lifecycle data, risiko keamanan, dan pengalaman pengguna.

Klasifikasi	CORE + SUPPORTING
Fokus silabus	Web API/Error handling/Interaction — Local Storage, Session Storage, custom error, extending Error, alert, prompt, dan confirm.
Project case	Data sensitif tersimpan di browser tanpa lifecycle, pesan error teknis tampil ke pengguna, dan dialog blocking mengganggu alur transaksi.
Bukti Intermediate	Storage inventory, expiry/cleanup test, custom error taxonomy, fallback UI, dan decision record interaksi.
Best practice	Simpan hanya data yang sesuai untuk browser, validasi ulang saat dibaca, dan pisahkan error teknis dari pesan aman untuk pengguna.

TARGET KOMPETENSI

- Memilih localStorage atau sessionStorage berdasarkan lifetime dan scope.
- Mendesain custom error serta error translation.
- Menggunakan alert/prompt/confirm hanya pada konteks yang tepat.

5.1 Web Storage

Aspek	localStorage	sessionStorage
Lifetime	Bertahan sampai dihapus	Berakhir bersama tab/session
Scope	Origin	Origin + tab
API	Sinkron; string key/value	Sinkron; string key/value
Contoh	Preferensi non-sensitif	Draft sementara per tab

Contoh 11. Versioned storage dengan fallback

```
const KEY = 'project-filter:v1';

export function saveFilter(filter) {
  localStorage.setItem(KEY, JSON.stringify({ ...filter, savedAt: Date.now() }));
}

export function loadFilter() {
  try { return JSON.parse(localStorage.getItem(KEY)) ?? null; }
  catch { localStorage.removeItem(KEY); return null; }
}
```

Security: Web Storage dapat dibaca script pada origin yang sama. Jangan menyimpan password, secret, atau token sensitif hanya karena API mudah digunakan. Risiko XSS tetap relevan.

5.2 Custom error dan translation**Contoh 12. Error untuk program dan pesan untuk pengguna**

```
class AppError extends Error {
  constructor(message, { code, cause, details } = {}) {
    super(message, { cause });
    this.name = 'AppError';
    this.code = code;
    this.details = details;
  }
}

function toUserMessage(error) {
  if (error.code === 'PROJECT_NOT_FOUND') return 'Project tidak ditemukan.';
  return 'Terjadi gangguan. Silakan coba kembali.';
}
```

5.3 alert, prompt, dan confirm

Dialog bawaan memblokir interaksi dan sulit dikustomisasi. Ia dapat berguna untuk latihan, debugging cepat, atau konfirmasi sederhana pada tool internal; untuk alur produksi yang kompleks, gunakan komponen dialog accessible dan aksi yang dapat dibatalkan bila risiko tinggi.

PRAKTIK & EVIDENCE 5

Buat penyimpanan filter versi v1, migrasi data rusak, serta error mapper dari HTTP/domain error ke pesan pengguna. Bandingkan confirm bawaan dengan dialog aplikasi.

Definition of done: Storage tidak menyimpan secret; parse failure pulih; key berversi; technical error tetap tercatat tanpa membocorkan detail sensitif; dialog dapat digunakan keyboard.

CHECKPOINT Bab 5

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa perbedaan lifetime localStorage dan sessionStorage?

Mengapa Web Storage bukan tempat secret?

Mengapa pesan pengguna sebaiknya dipisahkan dari detail error teknis?

BAB 06

Testable JavaScript dengan Jest dan Cypress

Menguji behavior serta kontrak penting tanpa membuat test rapuh terhadap detail implementasi.

Klasifikasi	CORE
Fokus silabus	Unit Test — Jest Unit Test dan Cypress Component Testing.
Project case	Test hijau tetapi bug tetap lolos karena hanya memeriksa implementasi internal dan tidak mencakup async, timer, error, atau perilaku komponen.
Bukti Intermediate	Test matrix, failing-first test, Jest report, Cypress evidence, dan regression test untuk bug nyata.
Best practice	Uji behavior pada boundary yang tepat; kontrol waktu dan dependency, serta hindari assertion yang bergantung pada detail implementasi rapuh.

TARGET KOMPETENSI

- Membedakan unit, integration, dan component test.
- Menulis Jest test dengan Arrange–Act–Assert dan dependency control.
- Menulis Cypress component test dari perspektif pengguna.

6.1 Test pyramid yang pragmatis

Level	Fokus	Contoh
Unit	Pure logic/contract kecil	risk score, mapper, validator
Integration	Boundary antarmodule/API	service + adapter
Component	UI behavior terisolasi	form, modal, loading/error
End-to-end	Critical user journey	login → update project

6.2 Jest unit test

Contoh 13. Table-driven unit test

```
export function riskLevel(score) {
  if (!Number.isFinite(score)) throw new TypeError('score must be finite');
  if (score >= 80) return 'critical';
  if (score >= 50) return 'warning';
  return 'normal';
}

test.each([[0, 'normal'], [50, 'warning'], [80, 'critical']])('riskLevel(%s) = %s', (score, expected) => {
  expect(riskLevel(score)).toBe(expected);
});
```

6.3 Async dan timer test

Contoh 14. Timer yang deterministik

```
jest.useFakeTimers();

test('debounce hanya menjalankan pencarian terakhir', async () => {
  const search = jest.fn();
  const debounced = debounce(search, 300);
  debounced('ne'); debounced('neuron');
  jest.advanceTimersByTime(300);
  expect(search).toHaveBeenCalledTimes(1);
  expect(search).toHaveBeenCalledWith('neuron');
});
```

6.4 Cypress component test

Contoh 15. Component behavior

```
describe('ProjectForm', () => {
  it('menampilkan error dan mempertahankan input', () => {
    cy.mount(<ProjectForm onSubmit={cy.stub().rejects(new Error('offline'))} />);
    cy.findByLabelText(/nama project/i).type('RUDIS');
    cy.findByRole('button', { name: /simpan/i }).click();
    cy.findByRole('alert').should('contain', 'Gagal menyimpan');
    cy.findByLabelText(/nama project/i).should('have.value', 'RUDIS');
  });
});
```

Test quality: Test yang baik gagal karena behavior penting rusak, bukan karena nama class, urutan internal function, atau snapshot besar berubah tanpa dampak pengguna.

PRAKTIK & EVIDENCE 6

Tulis unit test untuk mapper/validator dan component test untuk loading, error, retry, serta successful submit. Buat satu bug nyata tertangkap test.

Definition of done: Test deterministik; happy/error path teruji; dependency eksternal dikontrol; assertion berbasis output pengguna/contract; suite dapat dijalankan dari satu command.

CHECKPOINT Bab 6

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa yang membuat unit test cepat dan terisolasi?

Kapan fake timer diperlukan?

Mengapa selector berbasis role/label lebih kuat untuk component test?

BAB 07

TypeScript sebagai Kontrak

Menggunakan type system untuk memperjelas domain dan mencegah state tidak valid sebelum runtime.

Klasifikasi	CORE
Fokus silabus	TypeScript — types dan function types.
Project case	Payload API dianggap aman karena sudah diberi type, padahal data runtime tidak sesuai dan memicu state mustahil di UI.
Bukti Intermediate	Type model, function contract, narrowing, compile result, runtime validation boundary, dan negative test.
Best practice	Gunakan type untuk memperjelas kontrak compile-time; tetap validasi input eksternal pada runtime.

TARGET KOMPETENSI

- Menggunakan primitive, object, union, literal, optional, dan unknown secara tepat.
- Menerapkan narrowing serta discriminated union.
- Mendefinisikan function type, generic, dan boundary validation.

7.1 Type bukan validasi runtime

Contoh 16. Domain type dan runtime boundary

```
type ProjectStatus = 'draft' | 'active' | 'blocked' | 'done';

type Project = {
  id: string;
  name: string;
  status: ProjectStatus;
  progress?: number;
};

function parseProject(input: unknown): Project {
  // Validate unknown data at the boundary before asserting Project.
  return projectSchema.parse(input);
}
```

Penting: Respons API, localStorage, dan input pengguna tetap merupakan data runtime. TypeScript tidak memvalidasi data hanya karena kita menulis as Project.

7.2 Discriminated union

Contoh 17. State yang tidak ambigu

```
type LoadState<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; error: AppError };

function renderState(state: LoadState<Project[]>) {
  switch (state.status) {
    case 'success': return state.data.length;
    case 'error': return state.error.message;
    default: return null;
  }
}
```

7.3 Function type dan generic

Contoh 18. Kontrak function

```
type Fetcher = <T>(url: string, options?: RequestInit) => Promise<T>;
type ProjectPredicate = (project: Project) => boolean;

function filterBy<T>(items: readonly T[], predicate: (item: T) => boolean): T[] {
  return items.filter(predicate);
}
```

Gunakan	Untuk
unknown	Data belum dipercaya; wajib narrowing
never	State mustahil/exhaustive check

Gunakan	Untuk
readonly	Mencegah mutation melalui kontrak
generic	Relasi tipe yang tetap dipertahankan
union	Pilihan domain terbatas
any	Escape hatch sementara dan terukur, bukan default

PRAKTIK & EVIDENCE 7

Migrasikan module JavaScript project ke TypeScript strict. Bentuk union untuk async state, type API/domain terpisah, dan validation boundary.

Definition of done: Tidak ada any tanpa alasan; invalid state berkurang; external data divalidasi; function contract jelas; compile dan test lulus.

CHECKPOINT Bab 7

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

- Mengapa unknown lebih aman daripada any?
- Apa manfaat discriminated union untuk UI state?
- Mengapa as Type bukan validasi?

BAB 08

Separation of Concerns dan Service Layer

Memisahkan presentasi, orchestration, domain logic, dan akses data agar perubahan tidak menyebar ke seluruh aplikasi.

Klasifikasi	CORE
Fokus silabus	Code Structure dan React JS — separation of concerns serta Axios & Fetch API.
Project case	Komponen UI mengelola request, mapping payload, retry, error translation, dan business rule sekaligus sehingga sulit diuji dan diganti.
Bukti Intermediate	Dependency map, service boundary, mock/fake test, network evidence, serta keputusan Axios versus Fetch.
Best practice	Pisahkan UI, orchestration, business rule, dan transport berdasarkan alasan perubahan; pilih HTTP client dari kebutuhan, bukan kebiasaan.

TARGET KOMPETENSI

- Mendefinisikan ownership UI, hook/controller, domain, dan service.
- Mengisolasi Fetch/Axios serta mapping API.
- Menghindari god component dan duplicated effect.

8.1 Boundary yang dapat diuji

Layer	Tanggung jawab	Tidak seharusnya
Presentation	Render props dan emit event	Memanggil endpoint/mengerti payload mentah
Hook/controller	Orchestration state dan lifecycle	Menyimpan seluruh domain rule
Domain	Rule, validation, calculation	Bergantung pada DOM/framework
Service/adapter	HTTP, serialization, error translation	Mengatur visual state

Contoh 19. Service dan hook boundary

```
// services/projectService.ts
export const projectService = {
  async list(signal?: AbortSignal): Promise<Project[]> {
    const response = await http.get<ApiProject[]>('/projects', { signal });
    return response.data.map(toProject);
  }
};

// hooks/useProjects.ts
export function useProjects(service = projectService) {
  // Own loading/error/data lifecycle; expose commands to the UI.
}
```

8.2 Fetch versus Axios

Keduanya dapat menjadi HTTP client yang baik. Pilih berdasarkan kebutuhan pipeline project: interceptors, defaults, cancellation, error semantics, bundle, dan standarisasi tim. Bungkus client pada adapter agar komponen tidak bergantung langsung pada detail library.

Dumb component: Presentation component bukan berarti tanpa logika sama sekali. Ia boleh memiliki logika tampilan lokal; yang dihindari adalah business rule, data fetching, dan orchestration yang membuat komponen sulit digunakan ulang atau diuji.

RED FLAG — KESALAHAN UMUM

- useEffect memanggil endpoint di banyak komponen dengan pola berbeda.
- Komponen mengetahui DTO backend dan aturan domain sekaligus.
- Service mengubah state UI atau menampilkan toast langsung.
- Abstraksi generic dibuat sebelum ada pola penggunaan nyata.

PRAKTIK & EVIDENCE 8

Refactor god component: pindahkan mapper dan rule ke domain, HTTP ke service, orchestration ke custom hook, serta pertahankan UI sebagai presentation component.

Definition of done: Komponen dapat dites dengan props/mock service; tidak ada fetch langsung di presentation; error diterjemahkan pada boundary; dependency flow satu arah dan terdokumentasi.

CHECKPOINT Bab 8

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa ownership service layer?

Mengapa UI tidak sebaiknya memahami DTO backend?

Kapan custom hook layak diekstrak?

BAB 09

React Fundamentals dan Component Behavior

Mendesain component tree, props, state, composition, dan effect berdasarkan source of truth yang jelas.

Klasifikasi	CORE
Fokus silabus	React JS — component lifecycle, props/state, conditional rendering, composition, useState, dan useEffect.
Project case	Komponen menyimpan derived state, effect berulang tanpa perlu, dan conditional rendering menampilkan state yang saling bertentangan.
Bukti Intermediate	Render/effect trace, state model, component test, cleanup proof, dan diff hasil composition/refactor.
Best practice	Simpan state minimum, turunkan data saat render, gunakan effect hanya untuk sinkronisasi dengan sistem eksternal, dan utamakan composition.

TARGET KOMPETENSI

- Membedakan props, state, derived value, dan external system.
- Menggunakan composition serta conditional rendering tanpa state duplikat.
- Menulis useEffect dengan dependency dan cleanup yang benar.

9.1 Props, state, dan derived data

Data	Tempat	Contoh
Input dari parent	Props	project, permissions
Interaksi lokal berubah	State	selectedTab, draft
Dapat dihitung	Hitung saat render/memo bila perlu	filteredProjects
Berada di luar React	Effect/subscription	network, timer, DOM API

Contoh 20. Derived data dan composition

```
function ProjectList({ projects, filter, onSelect }) {
  const visible = projects.filter(project => matches(project, filter));
  if (visible.length === 0) return <EmptyState />;
  return (
    <ul>{visible.map(project =>
      <ProjectItem key={project.id} project={project} onSelect={onSelect} />
    )}</ul>
  );
}
```

9.2 Effect sebagai sinkronisasi**Contoh 21. Setup dan cleanup**

```
useEffect(() => {
  const controller = new AbortController();
  service.list(controller.signal)
    .then(setProjects)
    .catch(error => {
      if (error.name !== 'AbortError') setError(error);
    });
  return () => controller.abort();
}, [service]);
```

Mental model: Effect digunakan untuk menyinkronkan React dengan external system. Jangan memakai effect untuk menghitung data yang dapat diperoleh langsung dari props/state pada render.

9.3 Component lifecycle dalam function component

Pikirkan lifecycle sebagai render/commit serta setup—cleanup effect, bukan meniru method class secara mekanis. Development Strict Mode dapat menjalankan setup/cleanup tambahan untuk membantu menemukan effect yang tidak idempotent; kode tidak boleh bergantung pada effect hanya berjalan sekali.

RED FLAG — KESALAHAN UMUM

- Menyimpan filtered data sebagai state kedua.
- Menggunakan index sebagai key pada list yang dapat berubah urutan.
- Memanggil setState saat render.
- Effect tanpa cleanup untuk request, listener, atau timer.

PRAKTIK & EVIDENCE 9

Bangun project list dengan filter, selection, empty/error/loading, dan detail panel. Audit setiap state: source of truth, derived, atau external.

Definition of done: Tidak ada duplicated state; key stabil; effect punya cleanup; presentation dapat dirender dari fixture; semua state utama teruji.

CHECKPOINT Bab 9

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Kapan nilai sebaiknya dihitung daripada disimpan sebagai state?

Apa fungsi cleanup effect?

Mengapa key list harus stabil?

BAB 10

React State, Context/Redux, dan Forms

Memilih state ownership serta form strategy yang cukup kuat tanpa menambah kompleksitas yang tidak diperlukan.

Klasifikasi	CORE
Fokus silabus	React JS — Context & Redux serta pembuatan form dengan React Hook Form.
Project case	Seluruh state dipindah ke global store, form melakukan render berlebihan, dan error server tidak kembali ke field yang tepat.
Bukti Intermediate	State ownership map, Redux/Context rationale, form state test, server-error mapping, dan render observation.
Best practice	Tempatkan state sedekat mungkin dengan pemiliknya; gunakan global state dan library form hanya saat kompleksitas nyata membenarkannya.

TARGET KOMPETENSI

- Memilih local state, lifted state, Context, atau Redux berdasarkan kebutuhan.
- Menggunakan Redux Toolkit untuk shared state yang kompleks.
- Membangun form dengan React Hook Form, validation, dan server error.

10.1 State placement

Pilihan	Gunakan ketika	Hindari ketika
Local state	Satu component/subtree	Banyak consumer jauh
Lift state	Beberapa sibling berbagi	Prop drilling sangat dalam
Context	Nilai global jarang berubah/terbatas	High-frequency complex update tanpa desain
Redux Toolkit	Shared state kompleks, traceability, async flow	State lokal sederhana

Contoh 22. Redux Toolkit slice

```
const projectSlice = createSlice({
  name: 'projects',
  initialState: { entities: {}, status: 'idle' },
  reducers: {
    projectUpdated(state, action) {
      state.entities[action.payload.id] = action.payload;
    }
  }
});
```

10.2 React Hook Form**Contoh 23. Submit dan server error**

```
function ProjectForm({ saveProject }) {
  const { register, handleSubmit, setError,
    formState: { errors, isSubmitting } } = useForm();

  const onSubmit = async data => {
    try { await saveProject(data); }
    catch (error) { setError('root.server', { message: toUserMessage(error) }); }
  };
  return <form onSubmit={handleSubmit(onSubmit)}>...</form>;
}
```

10.3 API state bukan selalu global state

Server state memiliki caching, freshness, retry, dan invalidation semantics yang berbeda dari client UI state. Jangan otomatis menyalin semua respons API ke Redux. Gunakan pola yang disepakati project dan simpan hanya state yang memiliki ownership jelas.

Form contract: Client validation membantu UX; server tetap memvalidasi, mengotorisasi, dan mengembalikan error yang dapat dipetakan ke field atau root form.

PRAKTIK & EVIDENCE 10

Bangun form create/update project dengan field validation, server error, loading, reset, dan unsaved-change warning. Buat decision record state placement.

Definition of done: Label/error accessible; double submit dicegah; server error tampil tepat; draft tidak hilang tanpa keputusan; Context/Redux dipilih berdasarkan bukti kebutuhan.

CHECKPOINT Bab 10

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Kapan Context cukup tanpa Redux?

Mengapa server state berbeda dari UI state?

Apa yang harus terjadi pada server validation error?

BAB 11

Next.js App Router, Rendering, Layout, dan Authentication

Menentukan boundary server–client serta routing dan authorization tanpa membocorkan data atau mengirim JavaScript yang tidak diperlukan.

Klasifikasi	CORE
Fokus silabus	Next.js — App Router, server/client rendering, layout, dan authentication.
Project case	Komponen diberi 'use client' secara luas, data sensitif bocor ke client bundle, layout memicu fetch ulang, dan route hanya disembunyikan di UI.
Bukti Intermediate	Route tree, boundary server/client, bundle/network observation, authz test, dan redirect/error-state evidence.
Best practice	Mulai dari Server Component, pindahkan boundary client hanya saat perlu interaktivitas, dan tegakkan authorization pada server/data boundary.

TARGET KOMPETENSI

- Menggunakan file conventions App Router dan nested layout.
- Membedakan Server dan Client Components berdasarkan capability.
- Menerapkan authentication, session, dan authorization pada server boundary.

11.1 Struktur App Router

Contoh 24. Route, layout, loading, error, dan handler

```
app/  
  layout.tsx  
  page.tsx  
  loading.tsx  
  error.tsx  
  projects/  
    page.tsx  
    [id]/  
      page.tsx  
      edit/page.tsx  
  api/projects/route.ts
```

11.2 Server dan Client Components

Boundary	Cocok untuk	Catatan
Server Component	Data access server, secret, render non-interaktif	Tidak memakai browser API/state hook
Client Component	Event, state, effect, browser API	Tambahkan 'use client' pada boundary sekecil perlu
Route Handler	HTTP endpoint dalam app	Validasi, authz, status, cache harus eksplisit

Contoh 25. Boundary server–client

```
// app/projects/page.tsx – Server Component
export default async function ProjectsPage() {
  const session = await requireSession();
  const projects = await listProjectsForUser(session.user.id);
  return <ProjectScreen initialProjects={projects} />;
}

// ProjectScreen.tsx
'use client';
export function ProjectScreen({ initialProjects }) { /* interaction */ }
```

11.3 Authentication dan authorization

Authentication membuktikan identitas; authorization menentukan aksi/resource yang boleh diakses.

Validasi session pada server sebelum mengambil atau mengubah data.

Terapkan authorization dekat data access dan mutation, bukan hanya menyembunyikan tombol.

Gunakan cookie/session mechanism aman sesuai library dan deployment project.

Jangan mengandalkan middleware atau client redirect sebagai satu-satunya protection.

Security gate: UI permission meningkatkan pengalaman pengguna, tetapi bukan security boundary. Request langsung tetap harus ditolak oleh server ketika user tidak berwenang.

PRAKTIK & EVIDENCE 11

Bangun route list/detail/edit project, shared layout, loading/error boundary, serta protected mutation. Dokumentasikan alasan setiap Client Component.

Definition of done: Route dan layout benar; JavaScript client tidak berlebihan; secret tidak terkirim; unauthenticated dan unauthorized berbeda; mutation memvalidasi session, permission, dan input.

CHECKPOINT Bab 11

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Kapan komponen membutuhkan 'use client'?

Apa beda authentication dan authorization?

Mengapa menyembunyikan tombol bukan proteksi?

BAB 12

Refactoring, Profiling, dan Mini-Project

Memprioritaskan perbaikan berdasarkan dampak bisnis, technical debt, serta bukti hotspot—bukan selera pribadi.

Klasifikasi	CORE — INTEGRATED VALIDATION
Fokus silabus	Critical Thinking & Algorithm — refactoring decision making serta code profiling/hotspot identification; mengintegrasikan seluruh topik JavaScript.
Project case	Tim ingin mengoptimasi dan merombak aplikasi berdasarkan opini, tanpa baseline, regression safety, atau bukti hotspot.
Bukti Intermediate	Baseline, profiler trace, prioritized refactor, automated tests, before-after metric, decision record, dan technical defense.
Best practice	Kunci behavior, ukur baseline, perbaiki bottleneck atau risiko bernilai tinggi, lalu buktikan dampak dan trade-off setelah perubahan.

TARGET KOMPETENSI

- Membuat refactoring decision record dengan risiko dan rollback.
- Membaca latency/CPU/runtime evidence sebelum optimasi.
- Mengintegrasikan seluruh kompetensi pada aplikasi Issue & Risk Tracker.

12.1 Prioritas refactoring

Faktor	Pertanyaan
Business impact	Apakah defect/latency menghambat user, revenue, SLA, atau delivery?
Change frequency	Seberapa sering area berubah dan menimbulkan rework?
Risk	Apa blast radius, dependency, dan data/security consequence?
Evidence	Bug trend, test gap, complexity, latency, CPU, atau incident apa yang tersedia?
Safety	Adakah characterization test, feature flag, monitoring, dan rollback?

12.2 Profiling dan hotspot

Definisikan symptom dan target metric.

Reproduce dengan data dan environment representatif.

Rekam baseline tanpa mengubah banyak variabel.

Identifikasi hotspot melalui profiler/trace/latency breakdown.

Perbaiki bottleneck yang dominan, bukan baris yang tampak rumit.

Ukur ulang serta periksa correctness, memory, dan regresi.

Contoh 26. Measurement sederhana sebelum profiler mendalam

```
performance.mark('filter:start');
const result = filterAndRank(projects, query);
performance.mark('filter:end');
performance.measure('filterAndRank', 'filter:start', 'filter:end');

console.table(performance.getEntriesByName('filterAndRank').map(entry => ({
  duration: entry.duration
})));
```

Anti-optimization: Mengganti algoritma tanpa baseline dapat menambah kompleksitas tetapi tidak memperbaiki bottleneck. Performance work harus mempertahankan correctness dan diukur pada kondisi representatif.

12.3 Mini-project — Project Issue & Risk Tracker

Bangun aplikasi untuk mencatat issue, risk, owner, severity, due date, status, dan activity. Gunakan TypeScript, React, dan Next.js App Router; service layer dapat memakai API simulasi atau backend yang disediakan mentor.

Minimum feature

- ☐ List, filter, sort, detail, create/update, dan optimistic/confirmed feedback yang jelas.
- ☐ Loading, empty, error, retry, cancellation, validation, unauthorized, dan not-found state.
- ☐ Draft filter non-sensitif di storage dengan key berversi.
- ☐ Service/domain/UI boundary dan Server/Client Component boundary terdokumentasi.
- ☐ Unit test domain/mapper dan component test form/list.

- ☐ Satu refactoring record serta satu profiling experiment dengan before–after evidence.

12.4 Evidence package

- Repository dan reproducible setup.
- Architecture/decision record.
- Test report dan coverage context—bukan angka tanpa makna.
- Runtime/network screenshot untuk async/error path.
- Profiling note dan perbandingan baseline.
- Known limitations, security note, dan next improvement.

12.5 Pertanyaan review

State mana yang lokal, shared, server, atau derived—mengapa?

Bagaimana request usang dibatalkan dan error diterjemahkan?

Apa boundary UI, hook, domain, dan service?

Apa yang mencegah unauthorized mutation?

Test mana yang paling bernilai dan defect apa yang ditangkap?

Apa bukti hotspot dan apakah optimasi memberi dampak?

Bagian legacy mana yang Anda refactor terlebih dahulu dan apa rollback-nya?

Gate kelulusan: Aplikasi yang berjalan tetapi tidak memiliki kontrak, test, error path, authorization, atau reasoning belum menunjukkan level Intermediate. Study case assessment final menggunakan kasus serta dataset baru.

PRAKTIK & EVIDENCE 12

Selesaikan tracker, lakukan peer review, perbaiki minimal dua temuan, dan presentasikan runtime trace, test, architecture boundary, serta profiling evidence.

Definition of done: Build dapat direproduksi; minimum feature terpenuhi; critical path teruji; server authorization tersedia; peserta dapat menjelaskan trade-off dan bukti tanpa bergantung pada hafalan.

CHECKPOINT Bab 12

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa bukti minimum sebuah optimasi berhasil?

Mengapa characterization test penting sebelum refactor?

Apa perbedaan mini-project belajar dan assessment final?

Lampiran A1 — Traceability Detail 35 Topik Silabus

AUDIT CAKUPAN			
Tabel berikut memetakan seluruh 35 entri topik JavaScript pada sheet Silabus Intermediate Programmer. Nomor mengikuti urutan topik pada domain JavaScript, bukan nomor soal.			
No.	Kategori	Topik silabus	Bab
1	More variables	Value vs reference	1
2	More variables	Destructuring assignments	1
3	More operators	Relational, bitwise, string, comma, and unary operators	2
4	More operators	Checking instance (instanceof)	2
5	More functions	Arrow functions	2
6	More functions	IIFE (immediately invoked function expressions)	2
7	More functions	Recursions and stack	2
8	Event	Browser event and binding	3
9	Event	Event delegation	3
10	Asynchronous JavaScript	setTimeout and setInterval	4
11	Asynchronous JavaScript	Callbacks	4
12	Asynchronous JavaScript	Promises	4
13	Asynchronous JavaScript	Async await	4
14	Web API	Storage (Local Storage, Session Storage)	5
15	Error handling	Custom errors, extending errors	5
16	Interaction	alert, prompt, and confirm	5
17	Unit Test	Jest Unit Test	6
18	Unit Test	Cypress Component Testing	6

No.	Kategori	Topik silabus	Bab
19	TypeScript	TypeScript types	7
20	TypeScript	TypeScript function types	7
21	Code Structure	Separation of Concerns (UI vs Logic)	8
22	React	Context & Redux	10
23	React	Create Form using React Hook Form	10
24	React	Component lifecycle	9
25	React	Props & state	9
26	React	Conditional rendering	9
27	React	Composition	9
28	React	Basic Hook, useState useEffect	9
29	React	Axios & Fetch API	8
30	Next.js	App Router / routing	11
31	Next.js	Server & Client Rendering	11
32	Next.js	Layout	11
33	Next.js	Authentication	11
34	Critical Thinking & Algorithm	Refactoring Decision Making	12
35	Critical Thinking & Algorithm	Code Profiling & Hotspot Identification	12

Lampiran A2 — Traceability Kelompok Silabus ke Modul

Silabus	Bab	Bukti
Value/reference, destructuring	1	Identity test dan immutable update
Operators, arrow, IIFE, recursion/stack	2	Prediction dan traversal
Browser event/binding/delegation	3	Delegated dynamic table
Timers, callbacks, promises, async/await	4	Cancelable live search
Storage, custom error, interaction	5	Versioned storage dan error mapper
Jest dan Cypress component test	6	Deterministic suite
TypeScript types/function types	7	Strict domain contract
Separation UI/logic/service	8	Refactored boundary
React props/state/render/composition/hooks	9	Project screen states
Context, Redux, React Hook Form, Axios/Fetch	10	State/form decision
Next App Router, server/client, layout, auth	11	Protected routed app
Refactoring decision dan profiling hotspot	12	Decision record dan evidence

Lampiran B — Rubrik Self-Check Intermediate

Dimensi	Belum memenuhi	Memenuhi	Kuat
Runtime	Trial-and-error	Menjelaskan identity/async flow	Memprediksi edge case/race
Reliability	Happy path saja	Error/cancel/cleanup tersedia	Failure mode terukur
Testing	Test detail internal	Behavior kritis teruji	Suite cepat dan risk-based
Type/contract	any/assertion dominan	Strict dan boundary validation	Domain state sulit invalid
Architecture	God component	Boundary dan ownership jelas	Trade-off dan dependency terukur

Dimensi	Belum memenuhi	Memenuhi	Kuat
Security	UI dianggap proteksi	Server authz/input validation	Threat/abuse case dipertimbangkan
Refactor/performa nce	Tanpa baseline	Evidence dan rollback	Impact before–after kuat

Interpretasi: Siap assessment bila seluruh dimensi minimal Memenuhi dan tidak ada blocker pada authorization, data integrity, critical error path, atau reproducible build.

Lampiran C — Kunci Quick Check

Nomor	Jawaban inti
1.1	const mengunci binding, bukan isi object.
1.2	Property top-level; nested reference tetap berbagi.
1.3	Saat nilai hasil destructuring adalah undefined.
2.1	?? hanya fallback null/undefined; juga pada falsy seperti 0.
2.2	Arrow memakai lexical this dan tidak memiliki dynamic this sendiri.
2.3	Base case dan progres menuju base case; batasi kedalaman/input.
3.1	target sumber awal; currentTarget pemilik listener.
3.2	Satu ancestor menangani child saat ini dan yang ditambahkan kemudian.
3.3	Tidak menghentikan propagation.
4.1	Callback masuk antrean dan menunggu stack/urutan event loop.
4.2	Saat butuh partial result atau kegagalan satu tidak boleh menggagalkan semua.
4.3	Request usang dihentikan/diabaikan agar tidak menimpa state baru.
5.1	local bertahan; session terikat tab/session.
5.2	Script pada origin, termasuk saat XSS, dapat membacanya.
5.3	UX aman tanpa membocorkan detail; log tetap berguna bagi developer.
6.1	Pure input/output, dependency terkontrol, tanpa I/O nyata.

Nomor	Jawaban inti
6.2	Saat behavior bergantung waktu/debounce/retry dan harus deterministik.
6.3	Merepresentasikan akses pengguna dan lebih tahan refactor styling.
7.1	unknown wajib dinarrow; any melewati pemeriksaan.
7.2	Setiap state membawa data yang valid dan dapat dinarrow.
7.3	Assertion hanya memberi tahu compiler, tidak memeriksa data runtime.
8.1	HTTP/serialization/mapping/error translation.
8.2	DTO berubah karena backend dan tidak selalu sama dengan model tampilan/domain.
8.3	Saat orchestration/lifecycle berulang atau perlu diuji/digunakan ulang.
9.1	Jika dapat dihitung langsung dari props/state dan tidak perlu source of truth baru.
9.2	Melepas subscription, listener, timer, atau request lama.
9.3	React dapat mempertahankan identity component/item saat urutan berubah.
10.1	Shared value sederhana/terbatas dan update tidak kompleks.
10.2	Memiliki freshness, cache, retry, invalidation, dan ownership server.
10.3	Dipetakan ke field/root, input dipertahankan, dan user diberi arah perbaikan.
11.1	Saat membutuhkan event, state/effect, atau browser API.
11.2	Authn identitas; authz hak terhadap aksi/resource.
11.3	Request langsung tetap dapat dibuat; server harus menolak.
12.1	Metric representatif before–after, correctness tetap, dan regresi teruji.
12.2	Mengunci behavior legacy sehingga perubahan aman dan terukur.
12.3	Mini-project melatih; assessment memvalidasi independen dengan kasus baru.

Lampiran D — Debugging & Profiling Log

Field	Catatan
Issue/impact	Gejala dan dampak pengguna/bisnis
Environment	Commit, browser/runtime, data, route, network
Reproduction	Langkah minimum dan expected/actual
Runtime trace	Stack, event order, request, state transition
Hypothesis	Dugaan terurut sebelum perubahan
Evidence	Test, console, Network, Performance, profiler
Root cause/hotspot	Mekanisme penyebab dominan
Fix	Perubahan dan ownership boundary
Before–after	Correctness, latency/CPU/memory bila relevan
Regression/rollback	Skenario ulang dan cara memulihkan

Lampiran E — Glosarium

Istilah	Definisi
Identity	Kesamaan reference object, bukan kesamaan isi.
Mutation	Perubahan data pada identity yang sama.
Closure	Function beserta lexical environment yang diingat.
Call stack	Tumpukan execution context function sinkron.
Promise	Representasi penyelesaian asynchronous di masa depan.
Race condition	Hasil bergantung urutan operasi yang tidak terjamin.
Cancellation	Menghentikan/menandai pekerjaan usang.
Narrowing	Mempersempit union TypeScript berdasarkan bukti runtime/control flow.

Istilah	Definisi
Derived state	Nilai yang dapat dihitung dari source of truth lain.
Server Component	Komponen Next.js yang dirender pada server dan tidak memakai API client.
Characterization test	Test yang merekam behavior legacy sebelum refactor.
Hotspot	Bagian yang dominan menyumbang biaya/latensi/risiko.

Lampiran F — Referensi Resmi

Gunakan dokumentasi yang sesuai dengan versi package lock project. Tanggal akses: 20 Agustus 2026.

- [MDN — JavaScript Guide](#)
- [MDN — Introduction to events](#)
- [MDN — Using promises](#)
- [MDN — Web Storage API](#)
- [MDN — Fetch API](#)
- [Jest — Getting started](#)
- [Cypress — Component testing](#)
- [TypeScript Handbook — Everyday types](#)
- [TypeScript Handbook — Functions](#)
- [React — Learn React](#)
- [React — Synchronizing with effects](#)
- [Redux Toolkit — Introduction](#)
- [React Hook Form — Get started](#)
- [Next.js — App Router](#)
- [Next.js — Server and Client Components](#)
- [Next.js — Authentication](#)